AI Agent Skills and MCP: Building the Right AI Agent Pattern

Stephen Simon

Chapter 2: Building AI Agent Skills

Creating Consistent Outputs with Skills

Test whether the launch-risk-review Skill holds up across inputs and wording

A team packages a process so that everyone gets results built on the same standard. In this video you test that claim the way a real team would: with a different week of data, with different people asking in different ways, and with a change to the standard itself.

Before you start

You need the `launch-risk-review` Skill from the previous video in `orbit-launch/.claude/skills/`. If you don't have it, run this from the folder that contains `orbit-launch` and `solutions`:

```
node solutions/catch-up.mjs 02_01
```

Then start Claude Code in `orbit-launch`, or start a new session if it's already running.

What's already in the workspace

- `inputs/status-update-week-39.md` is Maya's update from September 25. Earlier in the project, with less going wrong.
- `inputs/status-update-week-40.md` is her update from October 2, which you reviewed in the previous video.
- The Skill's `rubric.md` and `report-template.md` define the standard every review should follow.

What consistent means here

AI output isn't word-for-word repeatable, and it doesn't need to be. A review is consistent when these things hold every time:

- **The structure:** the same heading, an Overall status line, the same five columns in the same order, and a Missing information section.
- **The standard:** the same rubric is applied, so the same facts get the same severity.
- **The evidence:** every row points to a quote or an ID.

The wording of sentences and recommended actions can vary. Those three things should not.

Step 1: Review a different week

Send this request:

```
Review @inputs/status-update-week-39.md for launch risks.
```

Look for:

- **Overall status: On track.**
- The pen test booking rated **Medium**. It has no owner, which the rubric says is at least Medium. The week 39 update doesn't mention the vendor's lead time, so there's nothing to justify High yet.
- The single duplicate charge report rated **Low**.
- The pen test vendor's lead time listed under **Missing information**.

Notice what the agent didn't do: it didn't guess a lead time to make the risk sound bigger. The Skill's rules tell it to list missing information instead of inventing facts, and that rule applies no matter which week you review.

Step 2: Ask the way a teammate would

Clear the conversation first. In Claude Code, type `/clear`. Then send:

```
Anything that worries you in @inputs/status-update-week-40.md before
launch?
```

This request never mentions a launch review, severity, or a table. The agent should still load the Skill, because the request matches the situations in its description.

Step 3: Ask a third way

Clear the conversation again and send:

```
Give me the launch risk table for week 40. @inputs/status-update-week-
40.md
```

Step 4: Compare the reports

Put the three reports side by side and check:

1. Do all three use the same heading, status line, and columns, in the same order?
2. Do steps 2 and 3, which reviewed the same update, give the same Overall status? (Expected: **At risk**.)
3. Is the pen test **Medium** in week 39 and **High** in week 40? (Same rubric, different facts.)
4. Does every row in every report include evidence?
5. Is the dark mode request left out of both week 40 reports?

If all five hold, the Skill is doing its job. Different input changes the findings. Different wording changes nothing that matters.

Step 5: Change the standard in one place

Now the maintenance payoff. Suppose leadership wants every risk tied to a milestone.

1. Open `.claude/skills/launch-risk-review/report-template.md`.
2. In the table header, add a column named `Milestone` right after `Risk`, and add a matching cell to the example row.
3. Clear the conversation and run the week 39 request from step 1 again.

The report now includes a Milestone column. You changed one file, and every future review, for every person on the team, follows the new format. With the prompt-only workflow, you would have had to find and update every copy of the prompt.

When you're done, undo the change so your workspace matches the rest of the course. You can edit the file back, or run:

```
node solutions/catch-up.mjs 02_01
```

Why this works

- **The template fixes the structure.** The agent doesn't decide how the report should look. `report-template.md` does.
- **Explicit rules fix the judgment.** Rules with numbers, like "more than two weeks" or "three or more tickets open more than 7 days," leave little room for interpretation.
- **The description decouples the Skill from wording.** Because the agent chooses the Skill based on what the request is about, people don't need to phrase requests a special way to get the standard applied.
- **Rules cover the gaps.** "Don't invent facts, list what's missing" keeps results consistent even when the input is thin.

When results drift

If two runs on the same input disagree on something that matters, treat it as a signal that a rule is too vague, not as random noise. For example:

- If severity flips between High and Medium, add the date math or threshold that decides it to `rubric.md`.
- If a column or section goes missing, make the template more explicit, or state in `SKILL.md` that the template must be followed exactly.
- If the Skill isn't used at all for some requests, add those situations or words to the `description`.

Then test again on more than one input. A Skill that works on one document hasn't really been tested yet.

Tip: why clear between runs

Clearing the conversation means each run starts fresh, the way a teammate's session would. Without it, the agent can lean on the previous report and you'd be testing its memory of the conversation instead of the Skill.